

RUNBOOK — Deploy captadel.com (Captain Adel standalone)

SECTION 06-operations-it

TYPE runbook

STATUS **active**

OWNER Founder

UPDATED 2026-06-16

Note (2026-06-13): Captain Adel now lives in its own repo, [iflygaca/Captain-Adel](#). Run these steps from that repo (its root is the former `captadel/`); a `captadel/...` path below maps to that root. Deploys now run from the repo's own `.github/workflows/deploy.yml` once `GCP_SA_KEY` is set.

Stand the Captain Adel service up on **captadel.com**: build the image, wire the model/embeddings credentials, deploy to a Kingdom region, map the domain, and connect the Fly GACA gateway. The one-command path is `captadel/deploy/deploy.sh`; everything below explains what it does and the manual steps around it.

Accounts, billing & quota (pilot subscriptions on captadel.com) are a separate layer with their own setup — a dedicated Captadel Firebase project, Stripe, and the GitHub Actions deploy. See `runbook-captadel-saas.md`. That layer ships dark, so this deploy works unchanged whether or not it is set up. For `firebase-admin` to use ADC, deploy this service **into the same GCP project** as the Captadel Firebase project.

PDPL (load-bearing). Real user questions are personal data and must be processed in-Kingdom. Deploy the service to a KSA region (me-central2 Dammam — target; me-central1 Doha — interim) and host the ALLaM model in-Kingdom for production. Hugging Face endpoints (US/EU) are great for **dev + evals** but are outside the Kingdom — fine for the public GACAR corpus (embeddings), not for the chat model on real traffic. See `runbook-pdpl-me-central2.md`.

0. Prerequisites (once)

```
gcloud --version                # install the Google Cloud SDK if missing
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
# billing must be ENABLED for Cloud Run:
gcloud billing projects describe "$(gcloud config get-value project)" --
format='value(billingEnabled)'
gcloud services enable run.googleapis.com artifactregistry.googleapis.com \
                           cloudbuild.googleapis.com secretmanager.googleapis.com
```

What you need on hand:

- `GEMINI_API_KEY` — required. Captain Adel's English path + the groundedness judge.
- `ADEL_API_KEY` — a shared secret you invent; the Fly GACA gateway sends it so its server-to-server calls skip the browser rate-limiter. Required to wire step 6.
- `ALLAM_BASE_URL` (+ `ALLAM_API_KEY`) — optional now. The in-Kingdom Arabic voice (OpenAI-compatible `/v1`). Without it, Arabic still answers via Gemini.
- `EMBEDDINGS_BASE_URL` (+ `EMBEDDINGS_API_KEY`) — optional now. Cross-lingual dense recall (BGE-M3). Without it, retrieval is BM25-only. See `captadel/.env.example` for the Hugging Face recipe.

1. Store the secrets

Either create them by hand:

```
printf '%s' "YOUR_GEMINI_KEY" | gcloud secrets create GEMINI_API_KEY --data-file=-
printf '%s' "YOUR_ADEL_KEY"   | gcloud secrets create ADEL_API_KEY   --data-file=-
# optional, when ready:
printf '%s' "https://xxx.endpoints.huggingface.cloud/v1" | gcloud secrets create
ALLAM_BASE_URL --data-file=-
printf '%s' "hf_xxx"          | gcloud secrets create ALLAM_API_KEY --data-file=-
printf '%s' "https://yyy.endpoints.huggingface.cloud/v1" | gcloud secrets create
EMBEDDINGS_BASE_URL --data-file=-
printf '%s' "hf_xxx"          | gcloud secrets create EMBEDDINGS_API_KEY --data-file=-
```

...or export them and let the script upsert:

```
export GEMINI_API_KEY=... ADEL_API_KEY=... ALLAM_BASE_URL=... ALLAM_API_KEY=...
cd captadel && ./deploy/deploy.sh secrets
```

The deploy wires **only the secrets that exist**, so a Gemini-only first deploy is fine — add the others later and re-run to light them up.

2. (Optional) Build the cross-lingual index — once

The dense index ships **inside the image** (it lives under `src/brain/`, picked up by the Dockerfile's `COPY . .`). Build it before deploying, pointing at your embeddings endpoint:

```
cd captadel
EMBEDDINGS_BASE_URL="https://yyy.endpoints.huggingface.cloud/v1" \
EMBEDDINGS_API_KEY="hf_xxx" \
npm run build:embeddings          # writes src/brain/_embeddings.json.gz
```

Skip this for a BM25-only first deploy; run it and redeploy when your endpoint is ready.

3. Deploy

```
cd captadel
./deploy/deploy.sh           # region defaults to me-central2 (Dammam)
# if the region is rejected for your project:
REGION=me-central1 ./deploy/deploy.sh  # interim (Doha)
```

The script builds from source, wires the present secrets, sets `MODEL_PROVIDER=auto` + `ARABIC_PROVIDER=allam`, runs `--min-instances 1` (keeps the BM25 index warm), and prints the service URL.

Verify:

```
curl -s https://captadel-xxxx.run.app/health # {status:ok ...; allam:true once ALLaM is wired}
```

Open the URL in a browser → the landing page should load **Arabic-first (RTL)**; the chat should stream a cited answer.

4. (If you split the repo) source

`captadel/` is developed as a git subtree here and has been split to `ifylgaca/Captain-Adel` (see `runbook-captadel-extraction.md`). Deploy works the same from either repo — the script only needs the `captadel/` tree with its `Dockerfile`.

5. Map captadel.com

Option A — Cloud Run domain mapping:

```
gcloud run domain-mappings create --service captadel --domain captadel.com --region me-central2
gcloud run domain-mappings create --service captadel --domain www.captadel.com --region me-central2
```

Then add the records it prints at your DNS host.

Option B — Cloudflare in front (you already use Cloudflare): add a `CNAME` for `captadel.com` (and `www`) to the `run.app` host, proxied. Keep TLS = Full (strict).

Either way, `config.js` already allow-lists `captadel.com` / `www.captadel.com` for CORS.

6. Wire the Fly GACA gateway to the live service

So the embedded chat (flygaca.com) proxies to the same brain:

```
# on the chat Cloud Function (Firebase) – see runbook-captain-adel.md:
ADEL_API_URL = https://captadel.com          # or the run.app URL
ADEL_API_KEY = <the same value as the ADEL_API_KEY secret>
```

Redeploy the `chat` function. The gateway sends `X-Adel-Api-Key` server-to-server.

7. Troubleshooting

Symptom	Fix
<code>LOCATION_POLICY_VIOLATED</code> / region rejected	Use <code>REGION=me-central1 ./deploy/deploy.sh</code> until me-central2 is granted.
<code>/health</code> shows <code>allam:false</code>	<code>ALLAM_BASE_URL</code> secret missing/empty, or the endpoint is down — Arabic falls back to Gemini.
Arabic answers but weak/irrelevant sources	Cross-lingual recall is off — build the index (§2) + set <code>EMBEDDINGS_BASE_URL</code> , redeploy.
First request slow / times out	Cold start builds the BM25 index; <code>--min-instances 1</code> (default) keeps one warm.
Embedded flygaca chat errors	<code>ADEL_API_URL</code> / <code>ADEL_API_KEY</code> not set on the <code>chat</code> function (step 6).

8. Rollback

```
gcloud run revisions list --service captadel --region me-central2
gcloud run services update-traffic captadel --region me-central2 --to-revisions
REVISION=100
```

9. Pre-launch quality gate (run with your keys)

<code>cd captadel</code>		
<code>GEMINI_API_KEY=...</code>	<code>npm run eval</code>	# English path, keyword
<code>gate</code>		
<code>GEMINI_API_KEY=... ALLAM_BASE_URL=...</code>	<code>npm run eval:allam</code>	# Arabic path
<code>GEMINI_API_KEY=... ALLAM_BASE_URL=...</code>	<code>npm run eval:parity</code>	# Arabic must match/beat
<code>EN, no regression</code>		

Green here = safe to point traffic at it.